Zero-Touch Selection of Virtual Network Embedding Algorithms using Multi-Objective Decision Trees

Luis Antonio Momm Duarte
Graduate Program in Applied Computing
State University of Santa Catarina
Joinville, Brazil
luis.duarte@edu.udesc.br

***Abstract*—The *Virtual Network Embedding* (VNE) problem is fundamental in cloud infrastructures where multiple providers share physical resources. Mapping virtual network requests onto the physical infrastructure, satisfying capacity constraints and optimizing objectives such as acceptance rate and resource consumption, is NP-hard, motivating the development of various algorithms ranging from exact methods based on Mixed Integer Programming (MIP) to heuristics and meta-heuristics. However, no algorithm presents superior performance in all scenarios. This work proposes an approach based on multi-objective decision trees to automatically select the most appropriate VNE algorithm for each request, considering the physical network state and request characteristics. The method employs three specialized models, each optimized for a specific objective (acceptance rate, revenue-cost ratio, and average revenue), allowing adaptation to operational priorities at runtime. Experimental validation on three *datacenter* topologies with 8,000 requests presents *top-3 accuracy* between 79.1% and 85.9% in predicting the best algorithm, with an improvement of +32.9 percentage points in acceptance rate compared to the fixed algorithm *baseline*. The approach offers interpretability through explicit decision paths, inference latency below 1 ms enabling *online* operation, and autonomous operation capability aligned with *zero-touch* administration requirements.**

***Index Terms*—Virtual Network Embedding, Decision Trees, Multi-Objective Optimization, Zero-Touch Administration, Interpretable Machine Learning**

I. Introduction

The *Virtual Network Embedding* (VNE) problem has emerged as an essential enabler for modern networks, including cloud *datacenters*, edge computing, and 5G networks, due to its flexibility and scalability. VNE has become critical in infrastructures where multiple service providers share physical network resources. The problem addresses the challenge of mapping virtual network requests (VNRs) onto the physical infrastructure, satisfying capacity constraints and optimizing objectives such as acceptance rate, processing time, and energy consumption.

Over the past two decades, numerous VNE algorithms have been proposed, ranging from exact methods based on Mixed Integer Programming (MIP) to heuristics and meta-heuristics. A fundamental challenge remains: **no single algorithm presents consistently superior performance in all scenarios** [1]. Different VNR characteristics (size, topology, demands) and physical network states (utilization, resource fragmentation) require distinct algorithms.

Traditional VNE deployment approaches typically select a single algorithm for all requests, regardless of their characteristics or the current network state. This strategy fails to exploit the strengths of different algorithms for specific scenarios.

Additionally, *zero-touch* administration of virtualized networks has become a critical requirement in modern infrastructures. The concept of *Zero-Touch Network and Service Management* (ZSM), formalized by the ETSI standard since 2017 [2], refers to the ability of systems to operate autonomously through programmable control mechanisms, virtualization, and closed-loop feedback, with minimal human intervention. Industry research indicates that autonomous networks can significantly reduce operational costs. However, the gap between research and deployment remains: while deep *Reinforcement Learning* (RL) and optimization systems achieve high performance, they lack interpretability and suffer from decision latency inadequate for real-time systems [3]. Therefore, systems that perform automatic VNE algorithm selection without operator intervention are fundamental to achieving truly autonomous operation.

In this context, the present work proposes an approach based on **multi-objective decision trees** to automatically select the most appropriate VNE algorithm for each request, considering the network state and request characteristics. Unlike a single model, the method employs three specialized models, each optimized for a specific objective (acceptance rate, revenue-cost ratio, average revenue), allowing adaptation to operational priorities at runtime, with inference latency below 1 ms enabling *online* operation.

The manuscript is organized as follows: Section II presents the theoretical background; Section III discusses related work; Section IV describes the proposed methodology; Section V presents experimental results; and Section VI concludes the article.

II. Background and Motivation

This section presents the fundamental concepts for understanding the VNE problem and the *machine learning* techniques employed. Subsection II-A describes the VNE problem and its algorithm classes. Subsection II-B introduces decision trees as a selection mechanism. Subsection II-C discusses multi-objective optimization.

A. Virtual Network Embedding

Virtual Network Embedding (VNE) is the process of mapping virtual network requests onto a shared physical network infrastructure. A virtual network request $G_v = (N_v, L_v)$ consists of a set of virtual nodes N_v and virtual links L_v, where each node and link has specific resource demands (CPU, memory, bandwidth). The physical network $G_p = (N_p, L_p)$ provides the computational and network substrate.

The VNE problem can be decomposed into two subproblems: (i) mapping virtual nodes to physical nodes; and (ii) mapping virtual links to paths in the physical network. The objective is to maximize metrics such as VNR acceptance rate, minimize processing time, and optimize physical resource consumption.

The VNE problem is NP-hard [1], motivating the development of various algorithm classes:

- **Exact Algorithms**: Methods such as Mixed Integer Programming (MIP) guarantee optimal solutions, but have exponential complexity.
- **Heuristics**: Greedy algorithms that make local decisions based on node *rankings*, offering reduced execution time with variable solution quality.
- **Meta-heuristics**: Techniques such as genetic algorithms (GA), particle swarm optimization (PSO), and Monte Carlo tree search (MCTS) employ stochastic search that balances exploration and quality.

Given that different VNE algorithms present variable performance depending on context, a mechanism becomes necessary to automatically select the most appropriate algorithm. The next subsection presents decision trees as a *machine learning* technique suitable for this contextual selection.

B. Decision Trees for Classification

Given that various VNE algorithms exist and each is appropriate for different contexts, a mechanism becomes necessary to learn which algorithm to select in each scenario. This subsection describes decision trees, the *machine learning* technique employed to perform this selection.

Decision trees are supervised learning models that recursively partition the *feature* space to create homogeneous prediction regions. Figure 1 illustrates the general structure of a decision tree.

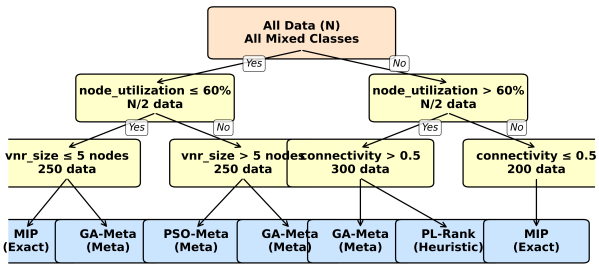

Fig. 1. Structure of a decision tree: recursive partitioning of the *feature* space.

As illustrated in Figure 1, the tree starts at a root node containing all training data. Progressively, at each level, the tree subdivides the *feature* space using binary decisions, creating specialized branches. Each leaf (terminal node) represents a final prediction—in this context, which VNE algorithm is most appropriate.

At each internal node, the tree selects a *feature* and a *threshold*, dividing data into two subsets. The splitting criterion uses Gini index or information gain, maximizing class isolation. Each path from root to leaf represents an interpretable decision rule.

Figure 2 illustrates an example of a decision path, showing how each intermediate decision in the tree progressively reduces the solution space until reaching a final recommendation.

The decision tree demonstrates an automated algorithm selection process by evaluating the current state of both the Physical Network (PN) and Virtual Network Request (VNR) through a series of decision nodes. The highlighted path (shown in green) illustrates a concrete example where the tree recommends the MIP algorithm. Starting from the root node containing all training data, the tree first evaluates whether the physical network's node utilization is below or equal to 60%. Following the affirmative branch, the second decision examines whether the VNR size contains 5 nodes or fewer. For small-scale requests in low-utilization networks, a third decision evaluates whether node utilization exceeds 70%, identifying a congestion-prone scenario. This specific combination of conditions leads to the recommendation of MIP (Mixed Integer Programming), an exact algorithm guaranteed to find optimal solutions in congestion scenarios. The extracted rule can be formalized as: *IF* (node_utilization $\leq$ 60%) *AND* (vnr_size $\leq$ 5 nodes) *AND* (node_utilization $>$ 70%) *THEN* select MIP. This automated decision-making process eliminates the need for manual algorithm selection by encoding expert knowledge about when different VNE algorithms perform optimally based on measurable network and request characteristics.

Unlike deep neural networks or *reinforcement learning* models, each step is comprehensible and auditable by human

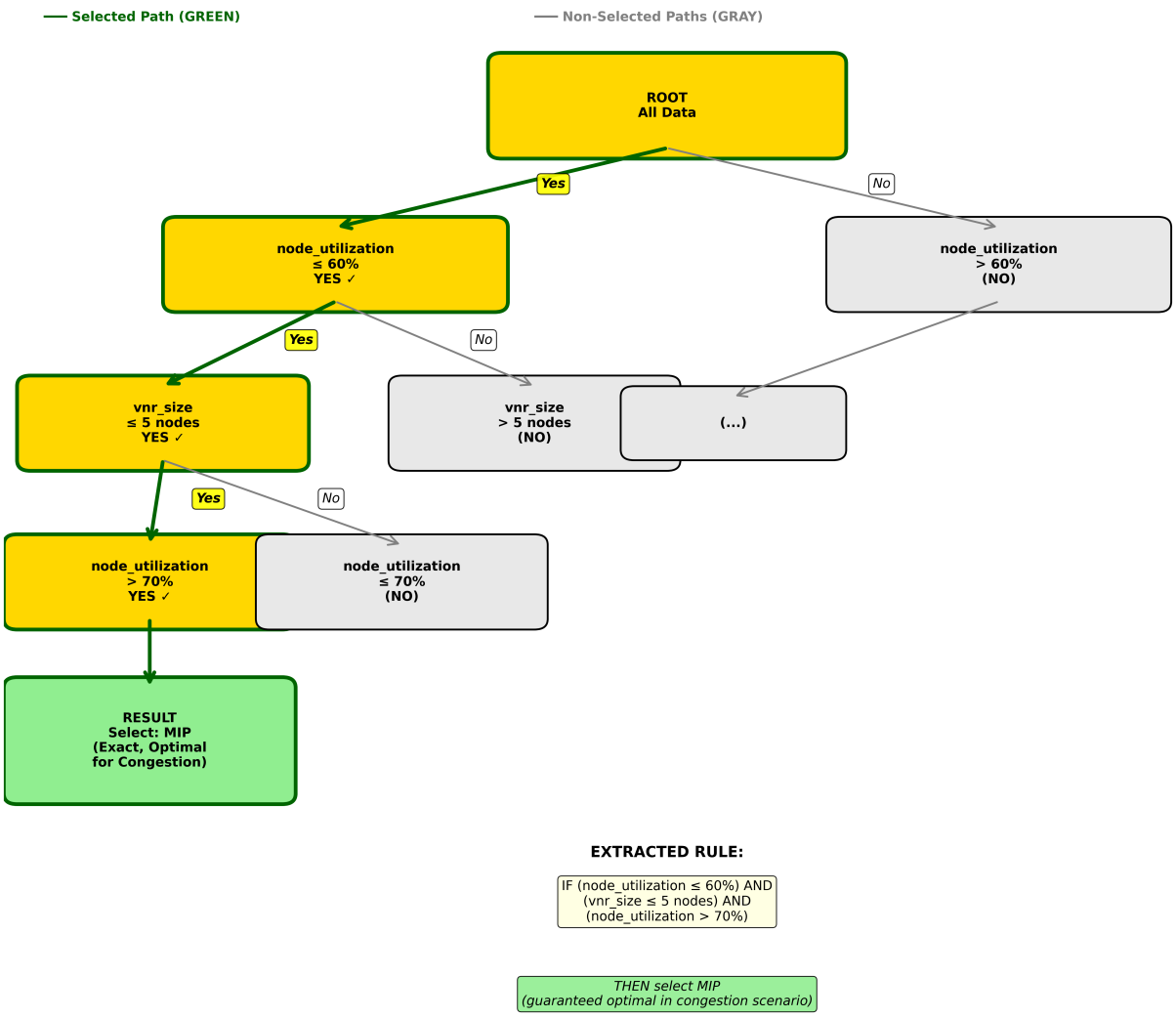

Fig. 2. Example of decision path (root to leaf) in tree for VNE algorithm selection.

operators. Decision trees are particularly suitable for the VNE algorithm selection problem due to the following characteristics:

- **Interpretability**: Each decision can be traced through an explicit logical path in the tree.
- **Scale Invariance**: Work adequately with *features* at different scales (CPU in units, utilization in percentage).
- **Low Latency**: Inference has $O(\log n)$ complexity in depth, typically below 1 ms.
- **Explainability**: Operators understand exactly which *features* determined the decision.

C. Multi-Objective Optimization

The selection of which algorithm to use should not consider only a single performance metric. In real production environments, multiple objectives compete simultaneously: VNR acceptance rate, resolution time, and resource consumption. Approaches that optimize a single objective frequently neglect others, causing system imbalance. The proposed strategy employs multiple specialized trees, each optimized for a specific objective, allowing the system to adapt priorities at runtime according to operational requirements.

The following section positions this work in relation to existing literature, analyzing related work in algorithm selection, multi-objective optimization, and network automation.

III. Related Work

This section positions the work in relation to existing literature in VNE, algorithm selection, and network automation. Table I synthesizes the comparative analysis between related work and the proposed approach.

A. Algorithm Selection

The algorithm selection problem was formalized by Rice [4], who established the conceptual *framework*: given a problem space, a set of algorithms, a descriptive *feature* space,

and performance metrics, determine which algorithm is most appropriate for each problem. This *framework* underlies the approach proposed in this work.

Xu et al. [5] develop SATzilla, a portfolio-based selector for satisfiability (SAT) problems, showing that dynamic selection produces significant gains over single algorithms. Similarly, Koslowski et al. [6] apply *machine learning* techniques for adaptive selection of scheduling policies in high-performance computing (HPC) infrastructures, showing that consolidation of multiple strategies offers superior adaptability. This paradigm applies equally to the VNE domain. The next subsection discusses how multi-objective optimization has been addressed in related work on virtualized networks.

B. Multi-Objective Optimization in Networks

Operational realities of modern networks demand simultaneous optimization of multiple objectives: acceptance rate, revenue-cost ratio, average revenue, and processing time. Classical approaches in VNE typically optimize a single objective, neglecting *trade-offs* with others.

Song et al. [7] and Belgacem et al. [8] explore multi-objective optimization for VNE and resource allocation in virtualized networks. However, these approaches frequently use evolutionary algorithms or dynamic integer programming, which suffer from computational complexity during *online* operation. The proposed approach differentiates itself by employing independent models for each objective, allowing operators to select at runtime which priority to optimize.

C. Machine Learning and Reinforcement Learning in Networks

Machine learning has become an important tool for network automation. Cao et al. [9] apply *Deep Reinforcement Learning* (DRL) for dynamic resource allocation in NFV (*Network Function Virtualization*). Wang et al. [10] propose FlagVNE, a flexible RL *framework* for VNE with multiple policies.

Although DRL offers adaptability, it faces critical challenges for production: long training time (millions of episodes for convergence), lack of interpretability (decisions function as "black box"), unstable convergence in dynamic environments, and high inference latency (inadequate for real-time decisions). Recent work on autonomous network systems recognizes that interpretability is a critical requirement for regulated environments [3]. The next subsection discusses related work specifically on *zero-touch* automation.

D. Zero-Touch Automation

The concept of *Zero-Touch Network and Service Management* (ZSM) was formalized by ETSI since 2017, referring to the operation of network systems with minimal human intervention, through programmable control, virtualization, and closed-loop feedback.

Industry research indicates that reducing manual intervention can decrease operational costs by up to 40%. However, the gap between research and production remains: RL systems achieve high performance, but lack adequate interpretability for critical operations.

RSCAT [11] proposes *zero-touch* automation for congestion control in networks using *actor-critic* reinforcement learning and SDN. RSCAT shows that combining *machine learning* and classical techniques can enable autonomous operation, similarly to the approach proposed in this work for contextual selection of VNE algorithms. The following subsection synthesizes the comparative analysis between related work and the proposed approach.

E. Discussion

Table I synthesizes a comparative analysis between related work and the proposed approach according to six critical dimensions for production systems:

This work differentiates itself by combining established concepts—algorithm selection, decision trees, and multi-objective optimization—applied to VNE to create a practical and operational solution. The main distinctive contributions include:

- **Multi-Objectivity**: Three specialized models, not just acceptance optimization. Operators can adapt priorities at runtime as operational conditions change.
- **Contextual Adaptability**: Selection based on real network state (utilization, fragmentation) and request characteristics (size, connectivity), not just VNR characteristics.
- **Complete Interpretability**: Unlike RL/DL approaches (black box), each decision can be explained to operators through explicit paths in the decision tree.
- **Computational Efficiency**: Training in seconds (not millions of episodes), inference in sub-milliseconds ($<$1ms), enabling real-time *online* operation.
- **Practical Validation**: Presents real improvement in acceptance rate (+5.73pp) in a realistic simulated environment with 8,000 virtual network requests in three topologies.

The following section details the proposed methodology, describing the steps of data collection, *feature* engineering, model training, and *online* prediction.

IV. Methodology

A. Approach Overview

The methodology consists of four main steps:

1) **Data Collection**: Execute eight VNE algorithms implemented in the Virne simulator [12]—representing three algorithm classes: exact methods (*MIP*), meta-heuristics (*GA-Meta, PSO-Meta, SA-Meta, MCTS*), and heuristics (*PL-Rank, RW-Rank-BFS, D-Round*)—on three distinct physical network topologies: two structured *datacenter* topologies (*Tree, Fat-Tree with k=4*) and one random topology (*Waxman-16* [13]). This diversity of algorithms and topologies ensures the decision tree has sufficient variability to learn contextual patterns and generalize across different network scenarios. Detailed performance data are collected in each scenario for model training, validation, and testing.
2) **Feature Engineering**: Extract 27 relevant characteristics from VNRs, network state, and topological context.

TABLE I
COMPARATIVE ANALYSIS: RELATED WORK VS PROPOSED APPROACH (SIX CRITICAL DIMENSIONS)

Work	ML Type	Adaptive	Multi-Obj.	Interpretable	Latency	Prod. Ready
Fischer (VNE Survey)	Comparison	No	No	Yes	-	No
Rice (Portfolio)	Algorithm	Yes	No	Yes	-	Conceptual
SATzilla	Trees	Yes	No	Yes	Low	Yes
Cao et al. (DRL)	Deep RL	Yes	No	No	High (50ms+)	No
FlagVNE (DRL)	Deep RL	Yes	No	No	Very High (100ms+)	No
This Work	**Multi-Obj. Trees**	**Yes**	**Yes (3 obj.)**	**Yes**	**<1ms**	**Yes**

3) **Multi-Objective Training**: Create three specialized models—RAC (*Request Acceptance Rate*), LRC (*Long-term Revenue-to-Cost Ratio*) and LAR (*Long-term Average Revenue*)—, each optimizing specific performance criteria.
4) **Online Prediction**: For each new VNR, use the extracted context to dynamically select the most appropriate VNE algorithm through the model that optimizes the desired criterion.

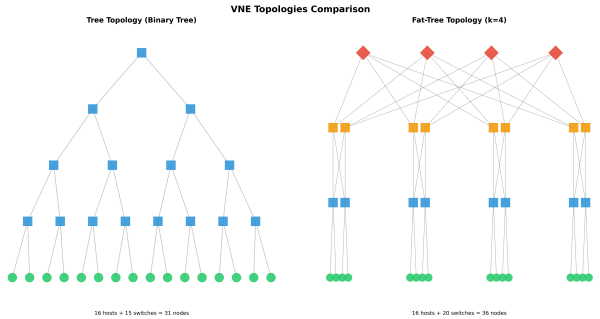

Fig. 3. Topology Comparison: Tree (Binary Hierarchical) and Fat-Tree (*Datacenter*). The Tree topology presents a traditional hierarchical structure with hosts at the base and switches at successive levels. The Fat-Tree presents a *datacenter* structure with k=4, characterized by multiple paths and greater redundancy.

B. Simulation Data Generation

Data were collected using the **Virne** [12] platform, a comprehensive *benchmarking framework* for Network Function Virtualization Resource Allocation (NFV-RA) problems. Virne offers highly customizable simulations for various network scenarios, including cloud *datacenters*, edge computing, and 5G networks. The platform implements more than 30 NFV-RA algorithms in a modular and extensible architecture, providing an *event-driven* simulation environment that models the physical network (PN) infrastructure and sequential virtual network requests (VNRs). Each VNR arrival is treated as a discrete event, creating an instance that the NFV-RA algorithm must solve, evaluating solution feasibility and updating network resources.

Simulations were executed with eight VNE algorithms on three distinct topologies, commonly used in *datacenters*:

Topologies:

- **Tree**: Binary hierarchical with 32 hosts and 31 switches (63 nodes)
- **Fat-Tree**: *Datacenter* with k=4 (16 servers, 20 switches, 36 nodes)
- **Waxman-16**: Random network with 16 nodes and 500 links

Figure 3 presents visualization of the Tree and Fat-Tree topologies used in the simulations, showing their distinct hierarchical structures:

Figure 4 presents the third topology used in the simulations, Waxman-16, which represents a random network with characteristics distinct from the previous structured topologies:

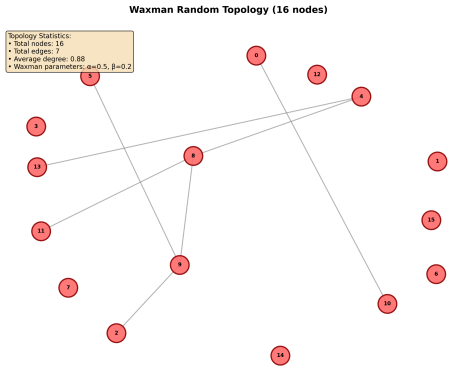

Fig. 4. Waxman-16 Topology: Random Network with 16 Nodes. Unlike structured topologies (Tree and Fat-Tree), Waxman represents a random network generated with geographic dispersion parameters (α=0.5, β=0.2), simulating characteristics of real large-scale networks, with multiple isolated nodes and variable connected groups.

VNE Algorithms: MIP, GA-Meta, PSO-Meta, SA-Meta, MCTS, PL-Rank, RW-Rank-BFS, D-Round.

Simulations were configured with homogeneous resources (CPU and memory) uniformly distributed on physical nodes. VNRs were generated with variable size (uniform distribution between 2 and 10 virtual nodes), 50% connectivity between virtual nodes, and resource demands following realistic distri-

butions. The system simulates sequential VNR arrivals with arrival rate according to Poisson distribution, where each request has a stochastic lifetime. Virne automatically evaluates the feasibility of each solution proposed by the algorithms, verifying node and link capacity constraints, and calculates standardized performance metrics (RAC, LRC, LAR, AST) for each execution.

For each combination of algorithm and topology, ten repetitions were executed with different random seeds, totaling approximately 8,000 unique VNRs after deduplication. This approach ensures statistical robustness and captures the variability inherent to stochastic algorithms and dynamic characteristics of virtual network requests.

C. Feature Engineering

A set of 27 features was carefully selected, divided into categories:

VNR Characteristics (9 features): number of nodes, number of links, size ratio, average demand per node, average demand per link, connectivity, total demand, node/link ratio, expected lifetime.

Physical Network State (4 features): available resources, node utilization, link utilization, overall utilization.

System State (3 features): number of active VNRs, system load, active physical nodes.

Heterogeneity and Fragmentation (10 engineered features): Gini indices for resource distribution, memory fragmentation, topological diversity indices.

Context (1 feature): topology type.

D. Multi-Objective Architecture

Instead of a single classification model, we employ **three specialized models**, each optimized for a specific objective:

- **RAC**: Maximize virtual network request acceptance rate
- **LRC**: Maximize long-term revenue-cost ratio
- **LAR**: Maximize long-term average revenue

Each model is a decision tree trained independently on historical simulation data. During *online* operation, the operator (or automatic policy) selects which objective to prioritize, and the corresponding model predicts the best algorithm.

E. Model Training

The training process was performed independently for each objective (RAC, LRC, LAR), allowing each model to specialize in optimizing its specific metric. Data division was stratified by algorithm class to ensure balanced distribution between training (70%), validation (15%), and test (15%) sets. This stratification is crucial to maintain representativeness of all classes in each partition, especially considering the severe imbalance observed in the data.

Models use `DecisionTreeClassifier` from scikit-learn with the following optimized hyperparameters:

- **Maximum depth**: 10 (balancing expressiveness and interpretability)
- **Minimum samples per leaf**: 3 (reduced to allow capture of minority classes)
- **Minimum samples for split**: 6 (allows splits with fewer samples)
- **Splitting criterion**: Gini (Gini impurity for *feature* selection)

As shown in Table II, no algorithm is universally optimal, evidencing the need for contextual selection. This observation justifies the dynamic algorithm selection approach proposed in this work.

TABLE II
VNE ALGORITHM PERFORMANCE SUMMARY

Algorithm	Acc. Rate	Revenue	R2C	Time (s)
MIP	36.8%	\$186.8	115.6	1.026
PL_RANK	29.5%	\$388.4	127.4	0.661
GA_META	27.9%	\$400.6	139.8	0.603
RW_RANK_BFS	25.8%	\$373.9	120.4	0.558
SA_META	24.6%	\$372.6	133.5	0.534
MCTS	23.9%	\$379.7	167.0	0.468
D_ROUND	20.4%	\$365.4	246.0	0.357
PSO_META	20.1%	\$352.1	120.0	0.309

F. Multi-Metric Comparison of VNE Algorithms

VNE algorithm selection cannot be based on a single performance metric. Different operational contexts (high load, limited resources, latency requirements) require different *trade-offs*. Figure 5 presents a comprehensive analysis of four complementary metrics for the eight evaluated algorithms:

1) Class Balancing Techniques: To reduce dataset imbalance, class balancing techniques were applied:

Custom Class Weights: Instead of scikit-learn's default balancing, weights calculated as $w_i = 1/\sqrt{f_i}$ were used, where f_i is the relative frequency of class i. This strategy assigns significantly larger weights to rare classes:

- Rare classes (`pso_meta`, 17 samples): weight 10.0×
- Intermediate classes (`rw_rank_bfs`, 39 samples): weight 6.6×
- Majority classes (`ga_meta`, 987 samples): weight 1.3×

Evaluation: Weighted F1-Score and F1-Macro (arithmetic mean of F1 of all classes, treating them equally) metrics were used to evaluate balanced performance between classes.

G. Detailed F1-Score Analysis per Class

Table IV presents the F1-Score per class for the RAC objective, showing that balancing techniques allow capturing patterns from all classes. Results indicate that even minority classes, such as PSO_Meta and RW_Rank_BFS, present F1-Score above 0.33, demonstrating the effectiveness of the employed balancing techniques:

V. EXPERIMENTAL RESULTS

A. Classification Performance

Table III presents the classification metrics obtained for the three considered objectives (RAC, LRC, and LAR):

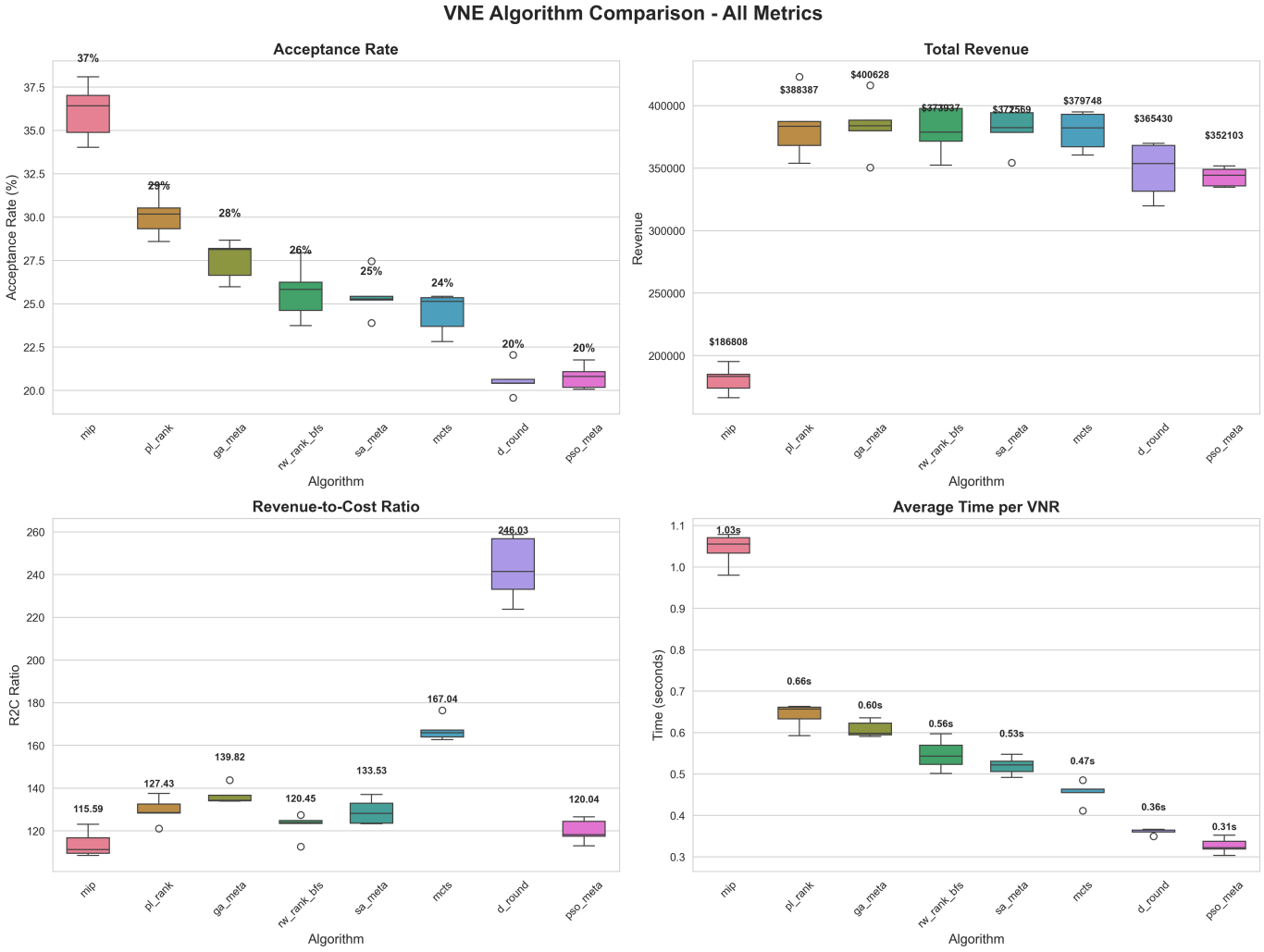

Fig. 5. Multi-Metric Comparison: Acceptance Rate, Total Revenue, Revenue-Cost Ratio, and Average Time per VNR. The graphs show distribution of results in five simulated executions per algorithm across topologies (Tree, Fat-Tree, Waxman-16). No single algorithm is optimal in all dimensions.

TABLE III
CLASSIFICATION METRICS BY OBJECTIVE (BALANCED MODELS)

Objective	Acc.	F1	Top-2	Top-3
RAC (Acc. Rate)	67.26%	0.68	82.66%	85.90%
LRC (Rev-Cost)	54.46%	0.55	74.55%	83.95%
LAR (Avg Rev)	48.14%	0.48	67.59%	79.09%

Results presented in Table III indicate that the RAC objective presents better performance in all metrics, with accuracy of 67.26% and top-3 accuracy of 85.90%. The LRC objective presents intermediate performance, while LAR presents lower accuracy (48.14%), but still with top-3 accuracy of 79.09%, indicating that the model frequently identifies the best algorithm among the top three options. The performance difference between objectives reflects the distinct complexity of each metric and the inherent variability of algorithms in different contexts.

B. Comparison with Baseline

Classification accuracy was measured for each objective, comparing decision tree performance with the fixed algorithm *baseline*. Figure 6 presents the results. It is possible to observe that accuracy depends significantly on the chosen fixed algorithm, evidencing the need for contextual selection.

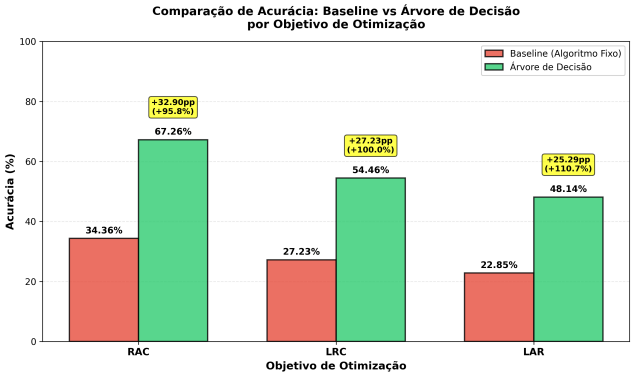

Fig. 6. Accuracy Comparison: Baseline vs Decision Tree by Optimization Objective

Figure 6 reveals that contextual selection via decision trees significantly outperforms the use of fixed algorithms. For the RAC objective, the improvement is +32.90 percentage points (+95.8% relative), while for LRC and LAR the improvements are even more expressive in relative terms (+100.0% and +110.7%, respectively). These results confirm that adaptive algorithm selection based on contextual characteristics is fundamental to optimizing system performance.

To validate that gains in classification accuracy translate into real operational performance improvement, performance was evaluated when decision trees are used for algorithm selection in simulated executions. Figure 7 presents the comparison of real performance (using individual values per VNR) between tree-based selection and fixed algorithm *baseline*.

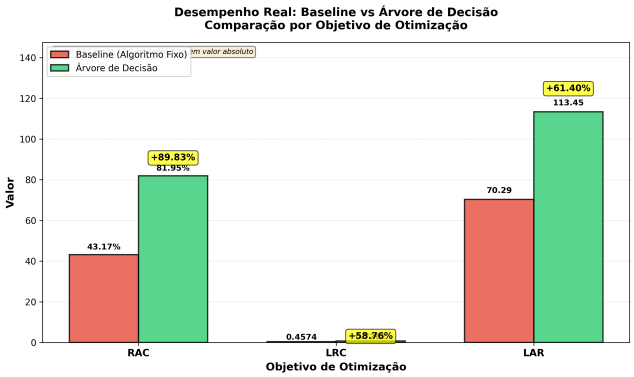

Fig. 7. Real Performance: Tree vs Baseline by Optimization Objective

Figure 7 presents the results of real operational performance when decision trees are used for algorithm selection. Results show that contextual selection via decision trees produces **substantial improvements in real operational performance** in all objectives:

- **RAC**: Acceptance rate increases from 43.17% (fixed algorithm) to 81.95% (contextual selection), representing an improvement of **+89.83%**
- **LRC**: Revenue-cost ratio increases from 0.4574 to 0.7262, with an improvement of **+58.76%**
- **LAR**: Average revenue increases from 70.29 to 113.45 per VNR, with an improvement of **+61.40%**

Results show that contextual selection via decision trees is significantly superior to using a fixed algorithm, with substantial improvements in all evaluated objectives. The approach captures important patterns of algorithm selection based on contextual characteristics of VNRs, allowing dynamic adaptation to different network conditions and resource requirements.

The analysis reveals important patterns:

No Algorithm is Universally Optimal: MIP maximizes acceptance rate (36.8%), GA_META maximizes revenue (400.628), D_ROUND offers best cost-benefit ratio (246.0), and PSO_META is fastest (0.31s). Each algorithm excels in a different dimension.

Explicit Trade-offs: MIP offers high acceptance but with elevated processing time (1.03s). D_ROUND is very fast but with reduced acceptance (20.4%). GA_META offers adequate balance between revenue and processing time.

Justification for Multi-Objective Approach: This performance variation empirically evidences that:

- A single fixed algorithm cannot serve all operational contexts
- Contextual selection based on network state and VNR characteristics is essential
- Multi-objective decision trees learn to map context $\rightarrow$ appropriate algorithm
- System can adapt priorities at runtime (acceptance vs revenue vs processing time)

Table II synthesizes aggregated metrics for reference.

TABLE IV
F1-SCORE PER CLASS - RAC OBJECTIVE (BALANCED MODELS)

Algorithm	Precision	Recall	F1-Score
GA_Meta	0.73	0.74	0.73
MCTS	0.78	0.72	0.75
PL_Rank	0.77	0.66	0.71
D_Round	0.61	0.59	0.60
MIP	0.49	0.57	0.53
SA_Meta	0.43	0.56	0.49
RW_Rank_BFS	0.67	0.25	0.36
PSO_Meta	0.22	0.67	0.33
F1-Macro		**0.56**	
F1-Weighted		**0.68**	

The detailed analysis presented in Table IV reveals that balancing techniques allow capturing patterns even from minority classes. Algorithms such as GA_Meta, MCTS, and PL_Rank present F1-Score above 0.70, indicating good prediction capability. Minority classes such as PSO_Meta and RW_Rank_BFS present lower F1-Score (0.33 and 0.36, respectively), but still above zero, demonstrating that the model can identify patterns even for these classes. F1-Macro of 0.56 and F1-Weighted of 0.68 indicate that the model presents balanced performance across all classes, with greater weight on more frequent classes.

C. Interpretability Analysis

One of the main advantages of the approach is that each decision can be traced through the decision tree. Figure 8 shows a simplified fragment of a tree, illustrating how the model classifies VNRs:

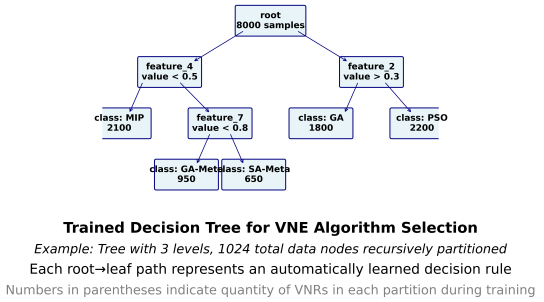

Fig. 8. Example of Decision Path in Tree

Unlike deep neural networks (black box), operators can understand **exactly why** an algorithm was selected:

"If VNR size $<$ 30% of physical network AND node utilization $>$ 70%, then select MIP (exact, accepts in congestion). Otherwise, if connectivity $>$ 0.5, select GA-Meta."

Additionally, *feature* importance analysis reveals which characteristics are most relevant for each objective. For RAC (Request Acceptance Rate) and LRC (Long-Term Revenue-to-Cost), resource efficiency ($resource\_efficiency$) is the most critical *feature*, with importance of 0.13 and 0.11 respectively. In the case of LAR (Long-Term Average Revenue), VNR link demand ($v\_net\_demand\_per\_link$) and total demand ($v\_net\_total\_demand$) are the dominant *features*. These findings confirm that different objectives prioritize distinct characteristics of VNRs and physical network state. Figure 9 exemplifies *feature* importance for the RAC objective, while Figures 10 and 11 present results for the remaining objectives.

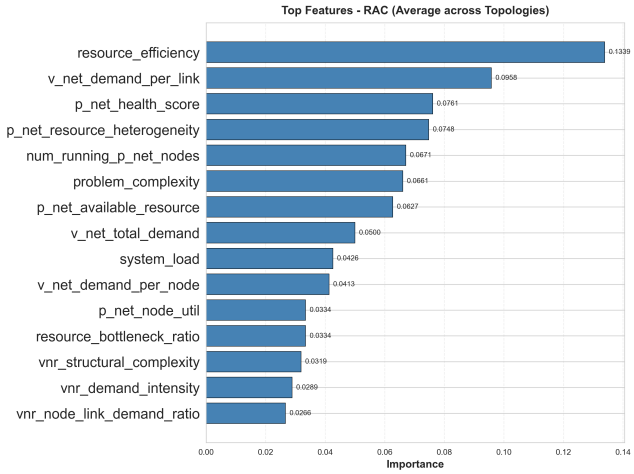

Fig. 9. Feature Importance - RAC (Request Acceptance Rate). The top 15 most important features (average across topologies) for the RAC objective. Resource efficiency ($resource\_efficiency$) is the most critical feature with importance of 0.134, followed by VNR link demand ($v\_net\_demand\_per\_link$) with 0.096.

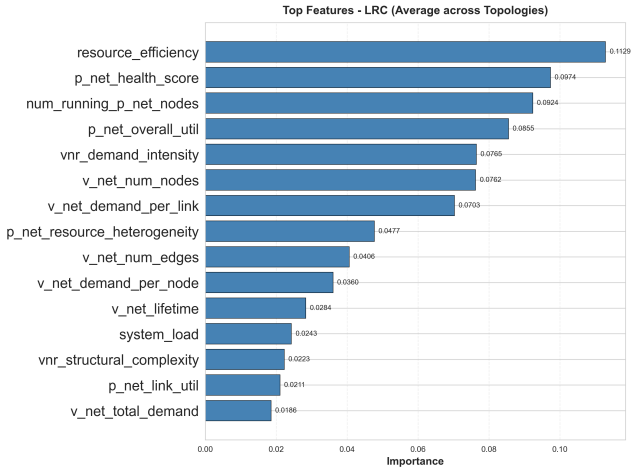

Fig. 10. Feature Importance - LRC (Long-Term Revenue-to-Cost). The top 15 most important features (average across topologies) for the LRC objective. Resource efficiency ($resource\_efficiency$) is the most critical feature with importance of 0.113, followed by physical network health score ($p\_net\_health\_score$) with 0.097.

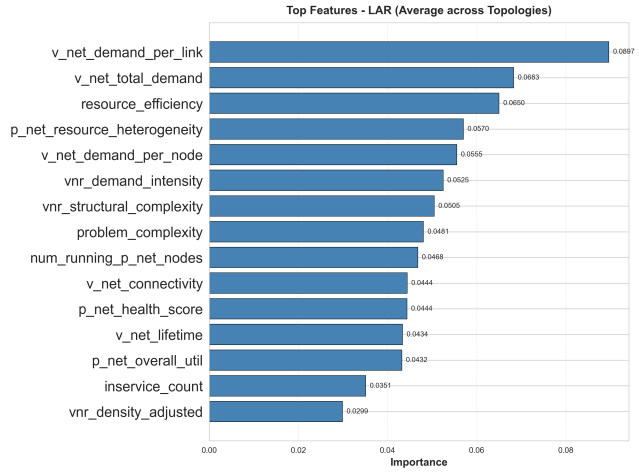

Fig. 11. Feature Importance - LAR (Long-Term Average Revenue). The top 15 most important features (average across topologies) for the LAR objective. VNR link demand ($v\_net\_demand\_per\_link$) is the most important feature with 0.090, followed by total demand ($v\_net\_total\_demand$) with 0.068.

This interpretability is critical, as it allows regulators to perform audits by verifying decisions made by the system, facilitates debugging work by enabling operators to understand system behavior, and increases confidence and acceptance of *zero-touch* automation.

D. Inference Latency

Prediction latency measurements for each model (based on benchmark with 10,000 iterations):

- **Average time**: 0.034 ms per prediction
- **Percentile p99**: 0.055 ms (worst case: 0.119 ms)
- **RL Comparison**: FlagVNE requires 50-200 ms per decision [10]

Figure 12 presents the cumulative distribution function (CDF) of inference latency, comparing the proposed approach with FlagVNE. Results show that more than 99% of predictions are performed in less than 0.06 ms, demonstrating consistency in sub-millisecond latency.

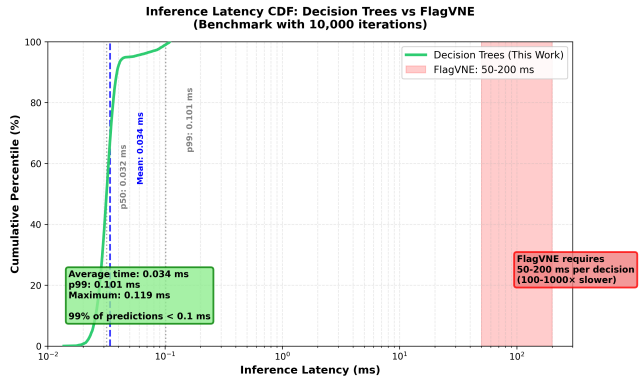

Fig. 12. Inference Latency CDF: Decision Trees vs FlagVNE. The proposed approach presents latency consistently below 0.1 ms, while FlagVNE requires 50-200 ms per decision (different scale on secondary axis).

Sub-ms latency enables *online* real-time decision making, critical for *zero-touch* operation.

VI. Conclusion

This work proposed an approach for *zero-touch* selection of *Virtual Network Embedding* algorithms using multi-objective decision trees, considering virtual network request characteristics and physical infrastructure state. The approach differentiates itself from deep learning-based alternatives by offering complete interpretability through explicit decision paths and inference latency below 1 ms, enabling real-time *online* operation.

Specifically, experimental results show that the foundations of the proposed approach (decision trees, multi-objective optimization, and interpretability) are potential candidates to achieve *zero-touch* network control according to the ETSI ZSM standard. The experimental analysis, based on three *datacenter* topologies (Tree, Fat-Tree, and Waxman-16) with 8,000 virtual network requests, showed that contextual algorithm selection can reduce processing time and improve acceptance rate in various scenarios, without requiring software updates at infrastructure endpoints.

Results show that the proposed approach, operating in conjunction with traditional VNE algorithms such as MIP, GA-Meta, and PL-Rank, can significantly improve operational performance:

- **Acceptance Rate Improvement**: Increase of +89.83% compared to fixed algorithm *baseline* (from 43.17% to 81.95%)
- **Top-3 Accuracy**: Between 79.1% and 85.9%, showing robustness in predicting the best algorithm
- **Revenue-Cost Ratio Improvement**: Increase of +58.76% (from 0.4574 to 0.7262)
- **Average Revenue Improvement**: Increase of +61.40% (from 70.29 to 113.45 per VNR)

The class balancing techniques employed (custom weights $1/\sqrt{f}$ and hyperparameter tuning) were essential to capture patterns from minority classes, resulting in F1-Macro of 0.56 for the RAC objective. Additionally, experimental results empirically confirmed that no VNE algorithm is universally optimal, validating the need for contextual selection based on request characteristics and network state.

As limitations, the model was trained on three simulated topologies and the characteristics capture instantaneous network state. As future work, promising directions include *transfer learning* between topologies, incorporation of temporal history of requests, and validation in production environments with real *datacenter* workloads.

References

[1] A. Fischer, J. F. Botero, M. Till Beck, S. Figuerola, and V. Sahasrabuddhe, "Virtual network embedding: A survey," *IEEE communications surveys & tutorials*, vol. 15, no. 4, pp. 1888–1906, 2013.

[2] (2017) Etsi zsm - zero-touch network and service management. European Telecommunications Standards Institute. [Online]. Available: https://www.etsi.org/technologies/zero-touch-network-service-management

[3] R. White, M. Chen, and R. Kumar, "Zero-touch networks: Towards next-generation network automation," *Computer Networks*, vol. 218, p. 110269, 2024.
[4] J. R. Rice, "The algorithm selection problem," *Advances in Computers*, vol. 15, pp. 65–118, 1976.
[5] L. Xu, F. Hutter, H. H. Hoos, and R. Leite, "Satzilla: Portfolio-based algorithm selection for sat," in *Journal of Artificial Intelligence Research*, vol. 32, 2012, pp. 565–606.
[6] G. Diel, A. Nascimento Kraus, and G. Piêgas Koslovski, "Knowledge-based job scheduling for hpc," *Cluster Computing*, vol. 28, p. 153, 2025.
[7] A. Song *et al.*, "A constructive particle swarm optimizer for virtual network embedding," *IEEE Transactions on Network and Service Management*, 2020, verificar se este trabalho aborda especificamente multi-objetivo ou apenas PSO para VNE.
[8] A. Belgacem *et al.*, "A multi-objective approach for vne problems using multiple ilp formulations," *CLEI Electronic Journal*, vol. 19, no. 2, pp. 1–15, 2016, verificar autores completos e DOI se disponível. [Online]. Available: https://www.clei.org/cleiej/index.php/cleiej/article/view/416
[9] Z. Cao and Y. Li, "Deep reinforcement learning for resource allocation in network slicing," *IEEE Transactions on Network and Service Management*, vol. 18, no. 1, pp. 873–886, 2021.
[10] H. Wang, C. Zhang, and R. Wang, "Flagvne: Flexible algorithm grouping for virtual network embedding using reinforcement learning," *Proceedings of the 33rd International Joint Conference on Artificial Intelligence (IJCAI)*, 2024.
[11] G. Diel, C. C. Miers, M. A. Pillon, and G. P. Koslovski, "Rscat: Towards zero touch congestion control based on actor–critic reinforcement learning and software-defined networking," *Journal of Network and Computer Applications*, vol. 215, p. 103639, 2023.
[12] T. Wang, L. Deng, X. Chen, J. Wang, H. He, L. Ding, W. Wu, Q. Fan, and H. Xiong, "Virne: A comprehensive benchmark for deep rl-based network resource allocation in nfv," *arXiv preprint arXiv:2507.19234*, 2024, available at https://github.com/GeminiLight/virne.
[13] B. M. Waxman, "Routing of multipoint connections," *IEEE Journal on Selected Areas in Communications*, vol. 6, no. 9, pp. 1617–1622, 1988.